

1. [2p] Dado el siguiente código Java,

- Dibujar un diagrama de clases que muestre todas las clases/interfaces involucradas (salvo Object) y sus relaciones. No es necesario incluir los atributos y métodos, a menos que sean relevantes para mostrar las relaciones entre las clases.
- Justificar: ¿La salida del programa es determinística?
- El `println` en la línea 16 de `Main.java` nunca se ejecuta. ¿Por qué? ¿En qué estado queda cada uno de los hilos?

Productor.java

```
1 public class Productor extends Thread {
2     ArrayBlockingQueue<String> queue;
3
4     public Productor(ArrayBlockingQueue<String> queue) {
5         this.queue = queue;
6     }
7
8     @Override
9     public void run() {
10        for (int i = 0; i < 5; i++) {
11            String item = "Item " + i + " desde " +
12                Thread.currentThread().getName();
13            queue.put(item);
14        }
15    }
```

Main.java

```
1 public class Main {
2     public static void main(String[] args) {
3         ArrayBlockingQueue<String> q = new
4             ArrayBlockingQueue<>(10);
5         Productor productor1 = new Productor(q);
6         Productor productor2 = new Productor(q);
7         Consumidor consumidor = new Consumidor(q);
8
9         productor1.start();
10        productor2.start();
11        consumidor.start();
12
13        productor1.join();
14        productor2.join();
15        consumidor.join();
16
17        System.out.println("Listo.");
18    }
```

Mundo.java

```
1 public class Consumidor extends Thread {
2     ArrayBlockingQueue<String> queue;
3
4     public Consumidor(ArrayBlockingQueue<String> queue) {
5         this.queue = queue;
6     }
7
8     @Override
9     public void run() {
10        while (true) {
11            String item = queue.take();
12            System.out.println("Consumido: " + item);
13        }
14    }
15 }
```

2. [1p] Considerar el siguiente código. Identificar un principio de diseño que se viola, indicar cuál es y justificar. Proponer una solución para evitar la violación del principio (no es necesario escribir el código).

```
1 public class App {
2     public void mostrarMenu(String usuario, String contrasena) {
3         Cuenta c = Cuenta.buscar(usuario);
4         if (c == null || !c.getContrasena().equals(contrasena)) {
5             System.out.println("Usuario o contraseña incorrectos");
6             return;
7         }
8         System.out.println("Bienvenido " + usuario);
9     }
10
11    public void mostrarDatosPersonales(String usuario, String contrasena) {
12        Cuenta c = Cuenta.buscar(usuario);
13        if (c == null || !c.getContrasena().equals(contrasena)) {
14            System.out.println("Usuario o contraseña incorrectos");
15            return;
16        }
17        System.out.println("Perfil de " + usuario + ":")
18        // ...
19    }
20 }
21
22 public class Cuenta {
23     public String getUsuario() { ... }
24     public String getContrasena() { ... }
25
26     public static Cuenta buscar(String usuario) { ... }
27 }
```

3. [2p] Escribir en Prolog la regla `repetido/2` que dado un valor y una lista determina si el valor aparece al menos 2 veces en la lista.

```
?- repetido(c, [a, b, c, d, b]).
false.
?- repetido(b, [a, b, c, d, b]).
true.
```

Se permite utilizar la regla estándar `member/2`:

```
?- member(d, [a, b, c]).
false.
?- member(b, [a, b, c]).
true.
```

4. [2p] Explicar en términos de programación funcional el funcionamiento del siguiente código en Clojure. Para cada una de las funciones involucradas, describir su propósito y cómo contribuye al resultado final, e indicar si son puras/impuras y si son de orden superior.

```
(reduce * (take 4 (repeat 2)))
;; resultado: 16
```

5. [2p] Dada la siguiente expresión lambda,

- Identificar las variables libres y ligadas.
- Mostrar los pasos de reducción usando orden normal (β -redex más externa desde la izquierda) hasta llegar a la forma normal (si existe). Aplicar reducciones α según sea necesario para evitar conflictos de nombres.

```
(( $\lambda x. (x (x w))$ ) (( $\lambda y. (y v)$ ) w))
```

6. [1p] Dado el siguiente código JavaFX, explicar brevemente el funcionamiento del programa, en términos de programación GUI y orientada a eventos. ¿Qué sucede cuando el usuario interactúa con la interfaz? ¿Cómo se logra este comportamiento?

```
public class App extends Application {
    @Override
    public void start(Stage stage) {
        Group root = new Group();
        Scene scene = new Scene(root, 200, 200);

        scene.setOnMouseClicked(e -> {
            Circle pelota = new Circle(10, Color.BLUE);
            pelota.setLayoutX(e.getSceneX());
            pelota.setLayoutY(e.getSceneY());

            root.getChildren().add(pelota);
            AnimationTimer t = new AnimationTimer() {
                double velocidad = 0;

                @Override
                public void handle(long l) {
                    if (pelota.getLayoutY() > scene.getHeight()) {
                        root.getChildren().remove(pelota);
                        this.stop();
                    }
                    pelota.setLayoutY(pelota.getLayoutY() + velocidad);

                    velocidad += 0.001;
                }
            };
            t.start();
        });

        stage.setScene(scene);
        stage.show();
    }
}
```